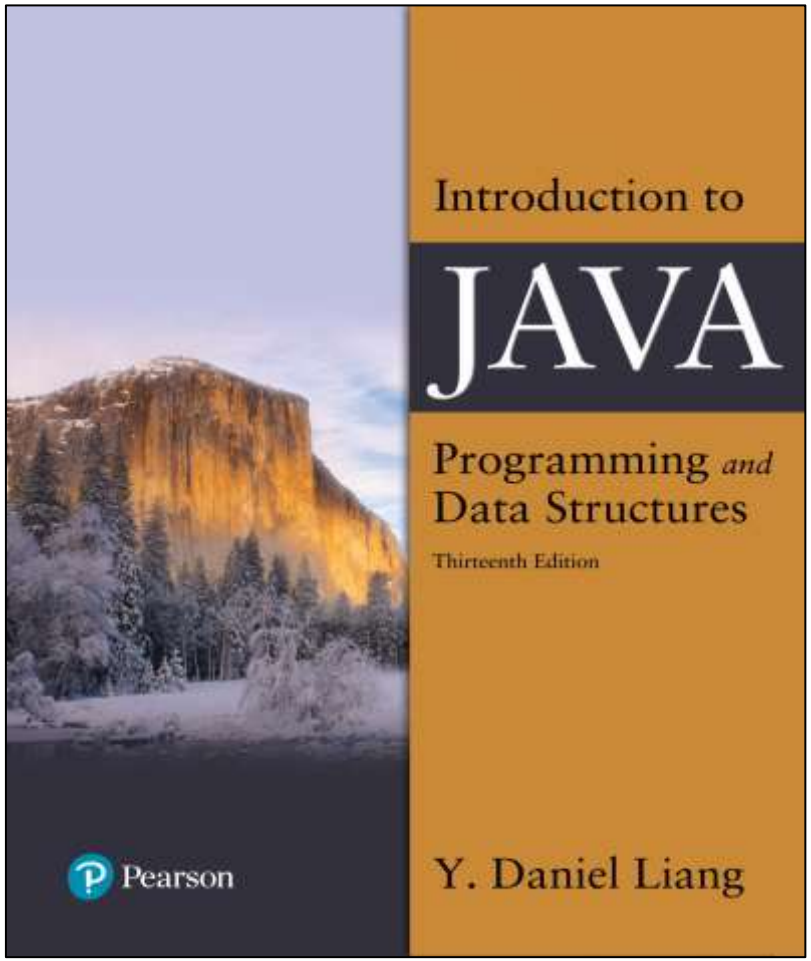
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 12

Exception Handling and Text IO

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations

When a program runs into a runtime error, the program terminates abnormally. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 4)

12.1 To get an overview of exceptions and exception handling (§12.2).

12.2 To explore the advantages of using exception handling (§12.2).

12.3 To distinguish exception types: **`Error`** (fatal) vs **`Exception`** (nonfatal) and checked versus unchecked (§12.3).

12.4 To declare exceptions in a method header (§12.4.1).

12.5 To throw exceptions in a method (§12.4.2).

12.6 To write a **`try-catch`** block to handle exceptions (§12.4.3).

12.7 To explain how an exception is propagated (§12.4.3).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 4)

12.8 To obtain information from an exception object (§12.4.4).

12.9 To develop applications with exception handling (§12.4.5).

12.10 To use the **`finally`** clause in a **`try-catch`** block (§12.5).

12.11 To use exceptions only for unexpected errors (§12.6).

12.12 To rethrow exceptions in a **`catch`** block (§12.7).

12.13 To create chained exceptions (§12.8).

12.14 To define custom exception classes (§12.9).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (3 of 4)

12.15 To discover file/directory properties, to delete and rename files/directories, and to create directories using the **`File`** class (§12.10).

12.16 To write data to a file using the **`PrintWriter`** class (§12.11.1).

12.17 To use try-with-resources to ensure that the resources are closed automatically (§12.11.2).

12.18 To read data from a file using the **`Scanner`** class (§12.11.3).

12.19 To understand how data is read using a **`Scanner`** (§12.11.4).

12.20 To develop a program that replaces text in a file (§12.11.5).

12.21 To read data from the Web (§12.12).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (4 of 4)

- **12.22** To develop a Web crawler (§12.13).

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Exception-Handling Overview

Show runtime error

Quotient

Fix it using an `if` statement

QuotientWithIf

With a method

QuotientWithMethod

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Exception Advantages

QuotientWithException

Now you see the **`advantages`** of using exception handling. It enables a method to throw an exception to its caller. Without this capability, a method must handle the exception or terminate the program.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Handling `InputMismatchException`

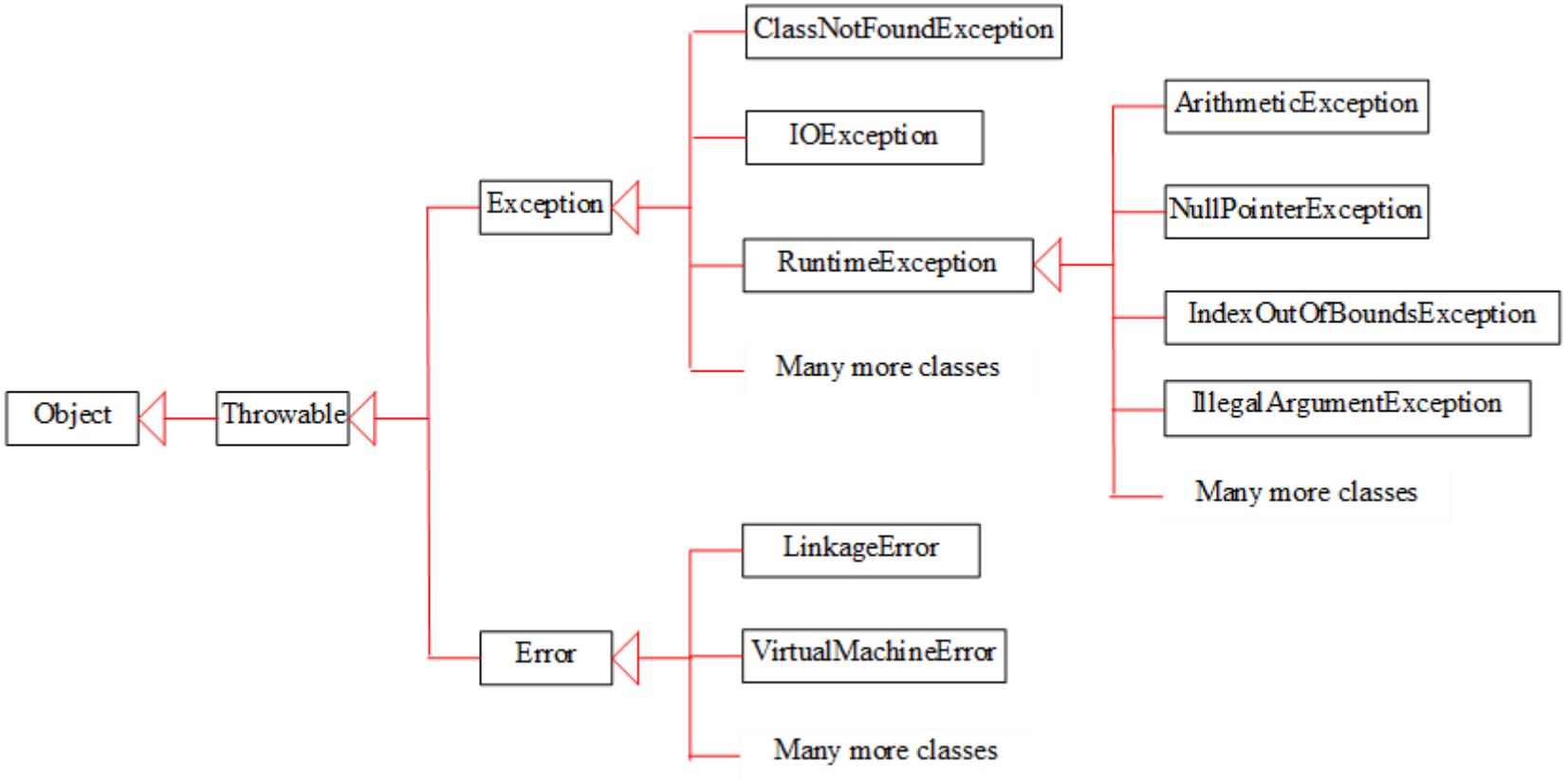
InputMismatchExceptionDemo

By handling `InputMismatchException`, your program will continuously read an input until it is correct.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

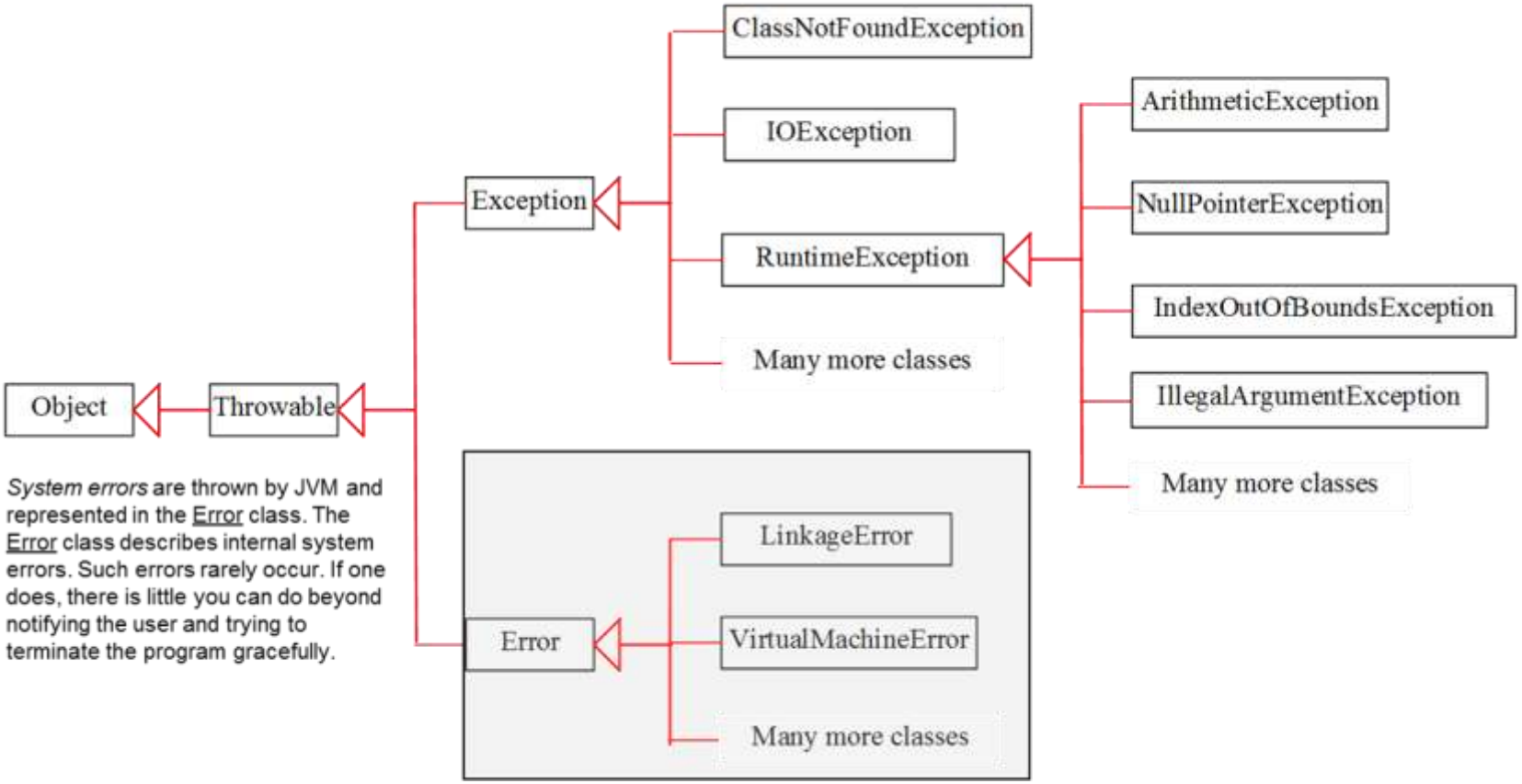
Exception Types



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

System Errors



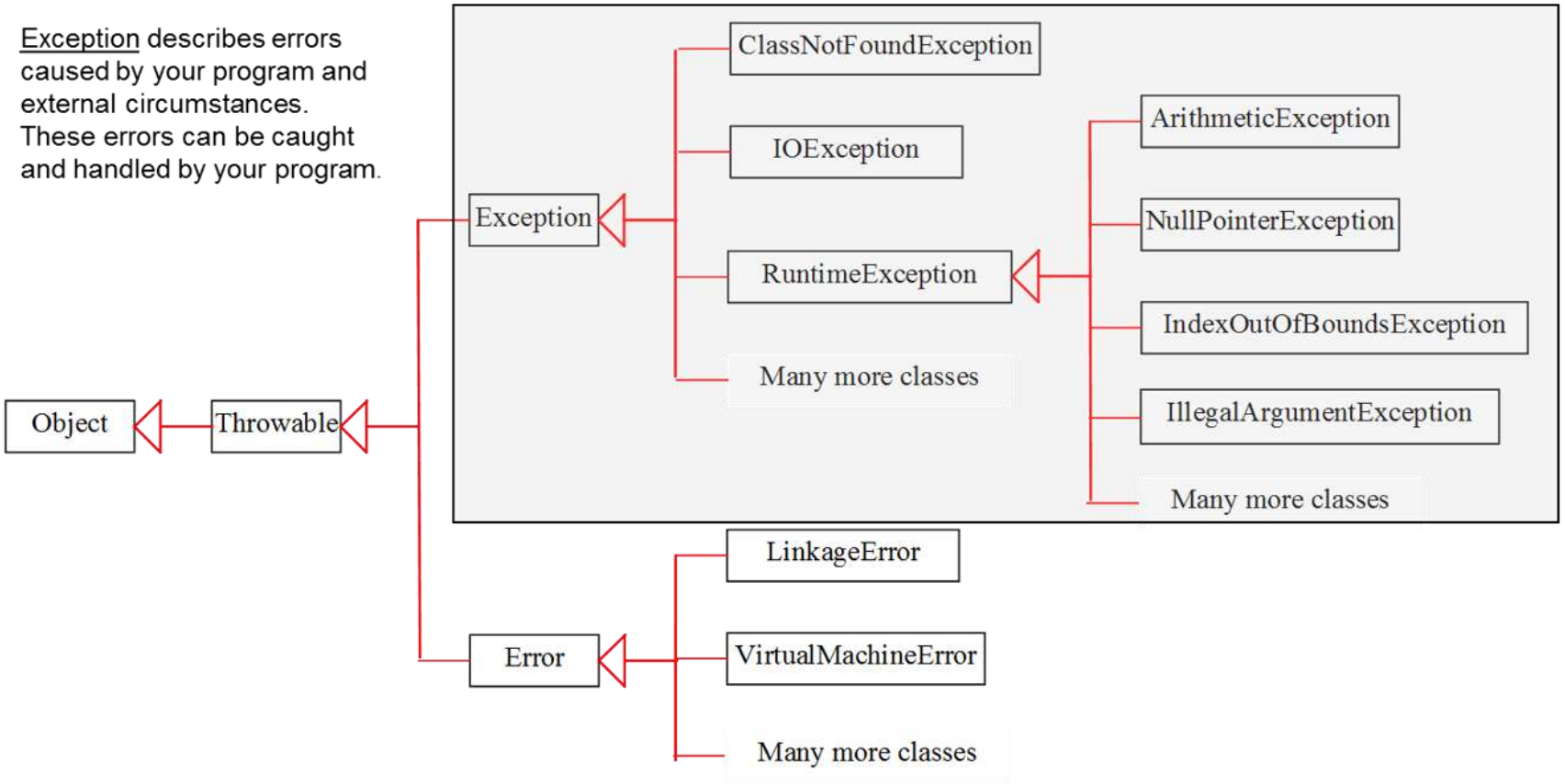
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Exceptions

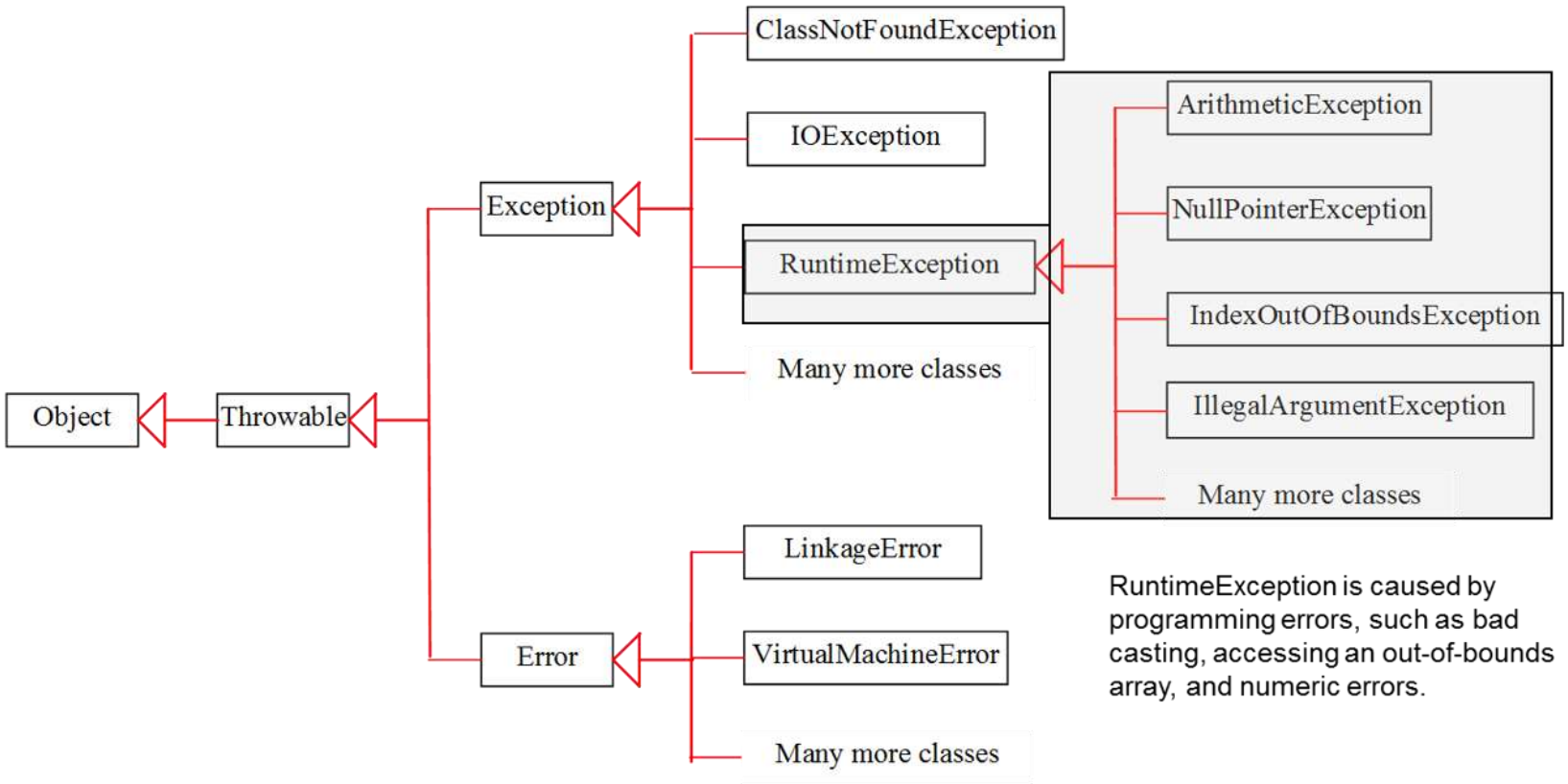
Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Runtime Exceptions



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Checked Exceptions vs Unchecked Exceptions

`RuntimeException`, **`Error`** and their subclasses are known as **unchecked exceptions**. All other exceptions are known as **checked exceptions**, meaning that the compiler forces the programmer to check and deal with the exceptions.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

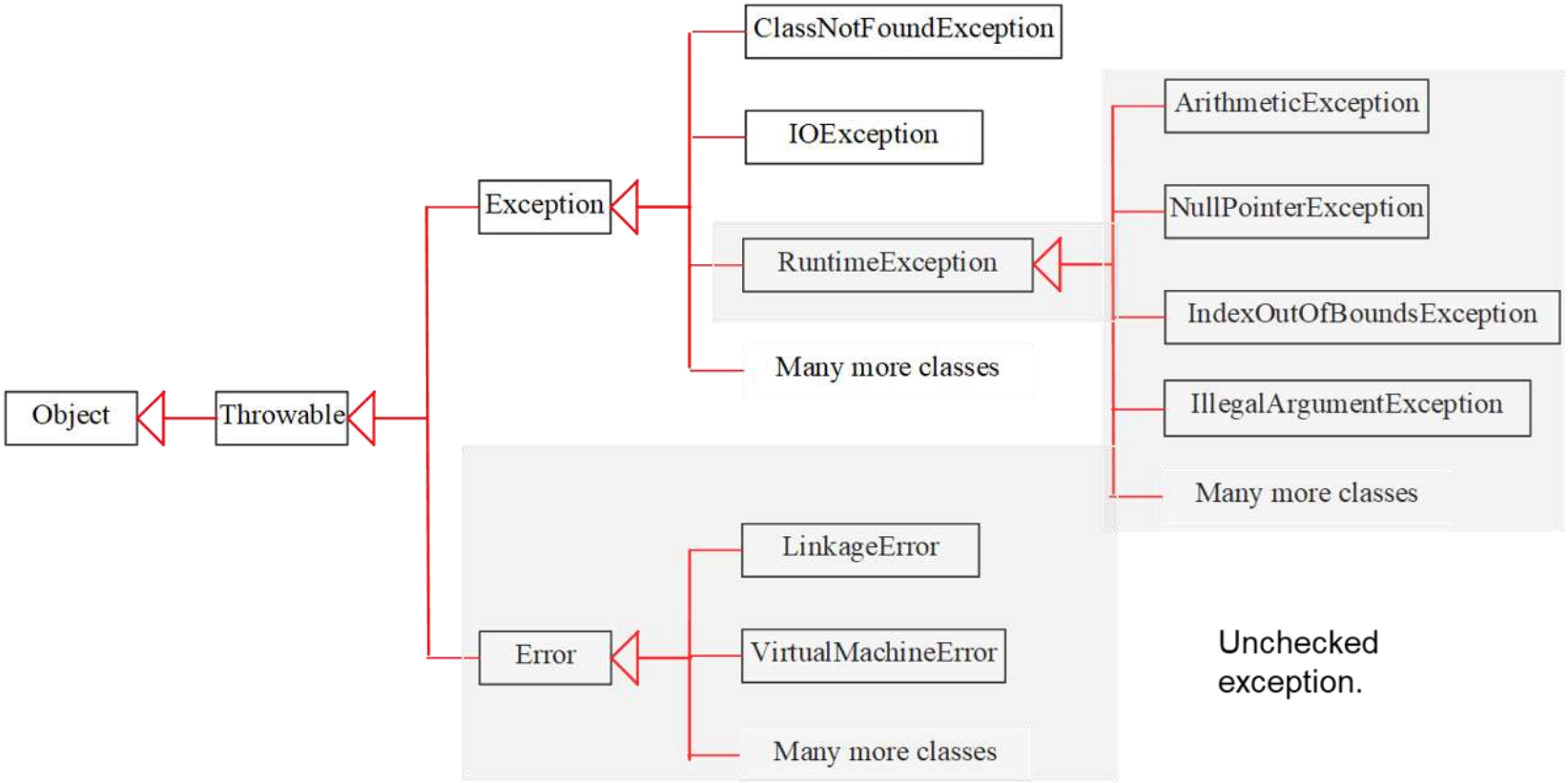
Unchecked Exceptions (1 of 2)

In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a **`NullPointerException`** is thrown if you access an object through a reference variable before an object is assigned to it; an **`IndexOutOfBoundsException`** is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in the program. To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Unchecked Exceptions (2 of 2)

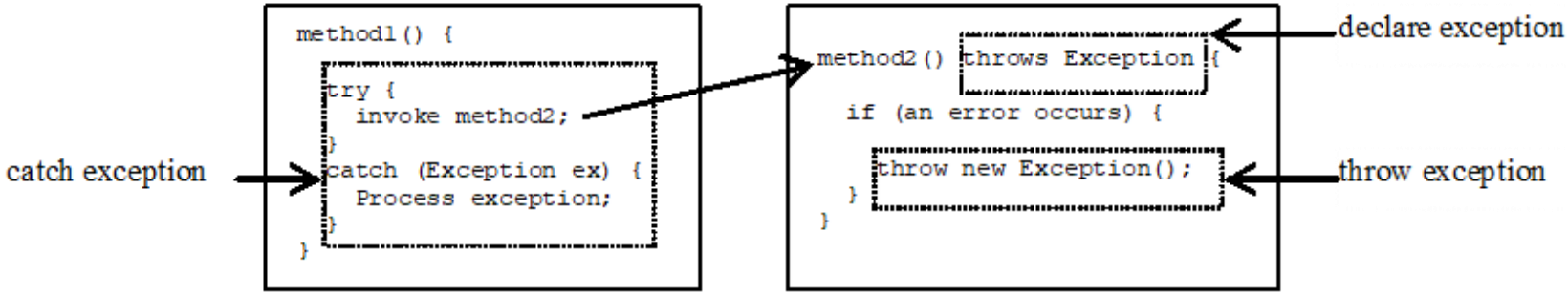


Unchecked exception.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Declaring, Throwing, and Catching Exceptions



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as **declaring exceptions**.

```
public void myMethod()
 throws IOException

public void myMethod()
 throws IOException, OtherException
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as **throwing an exception**. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Throwing Exceptions Example

```
  /** Set a new radius */
public void setRadius(double newRadius)
    throws IllegalArgumentException {
  if (newRadius >= 0)
    radius = newRadius;
  else
    throw new IllegalArgumentException(
      "Radius cannot be negative");
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

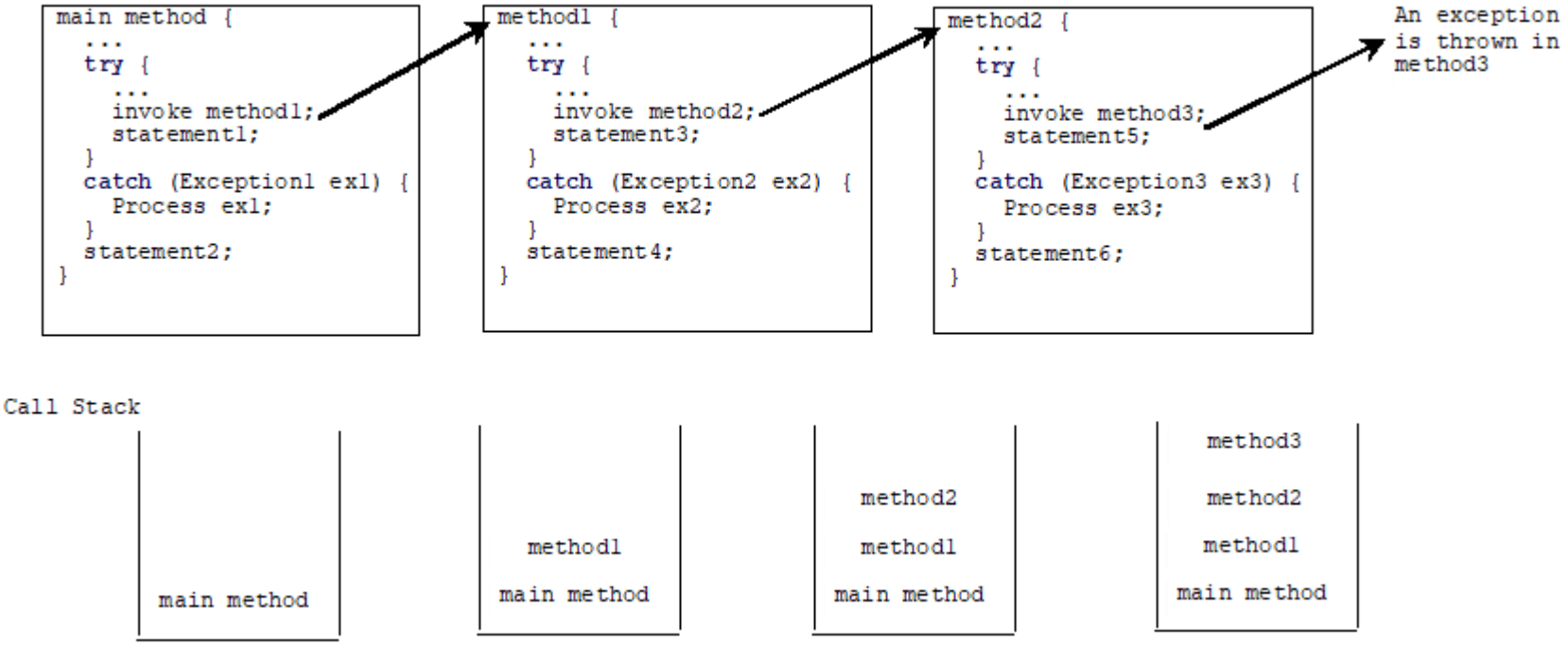
Catching Exceptions (1 of 2)

```
try {
  statements; // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Catching Exceptions (2 of 2)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Catch or Declare Checked Exceptions (1 of 2)

Suppose p2 is defined as follows:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Catch or Declare Checked Exceptions (2 of 2)

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Example: Declaring, Throwing, and Catching Exceptions

- Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the **`setRadius`** method in the Circle class defined in Chapter 9. The new **`setRadius`** method throws an exception if radius is negative.

CircleWithException

TestCircleWithException

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Rethrowing Exceptions

```
try {
  statements;
}
catch(TheException ex) {
  perform operations before exits;
  throw ex;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

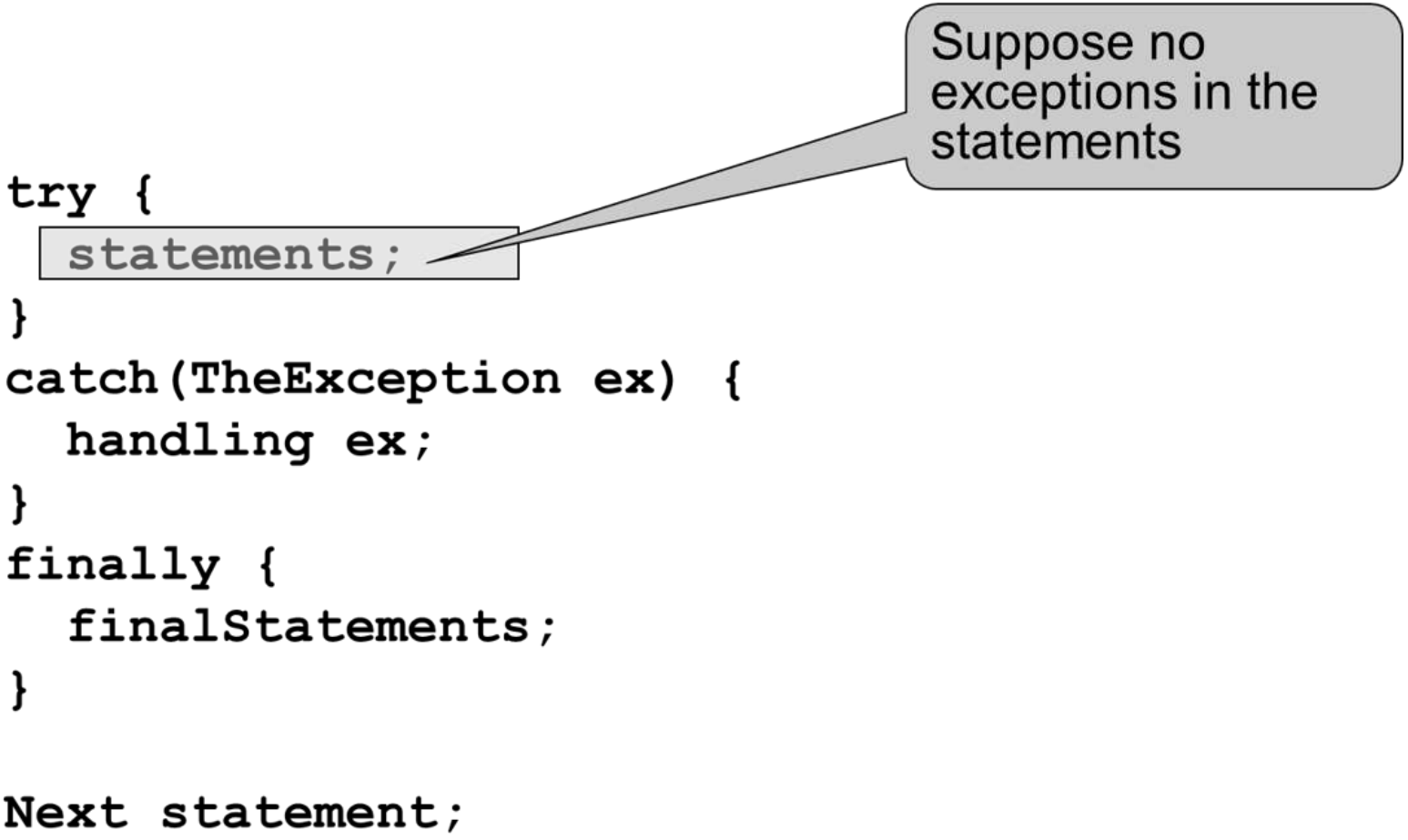
The `finally` Clause

```
try {
  statements;
}
catch(TheException ex){
  handling ex;
}
finally {
  finalStatements;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (1 of 11)



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (2 of 11)

```
try {
   statements;
}
catch(TheException ex) {
   handling ex;
}
finally {
   finalStatements;
}

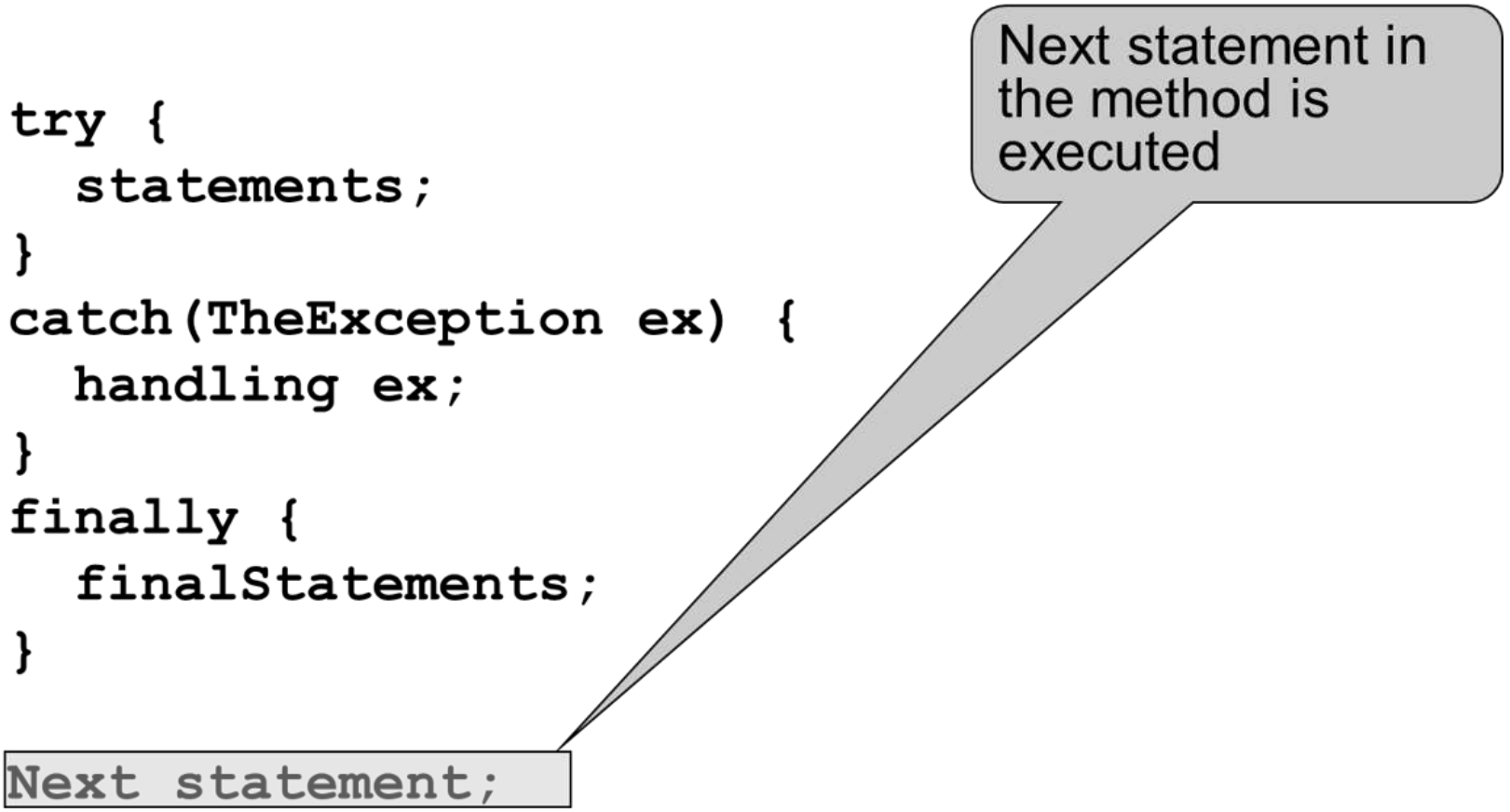
Next statement;
```

The final block is always executed

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (3 of 11)



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (4 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (5 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

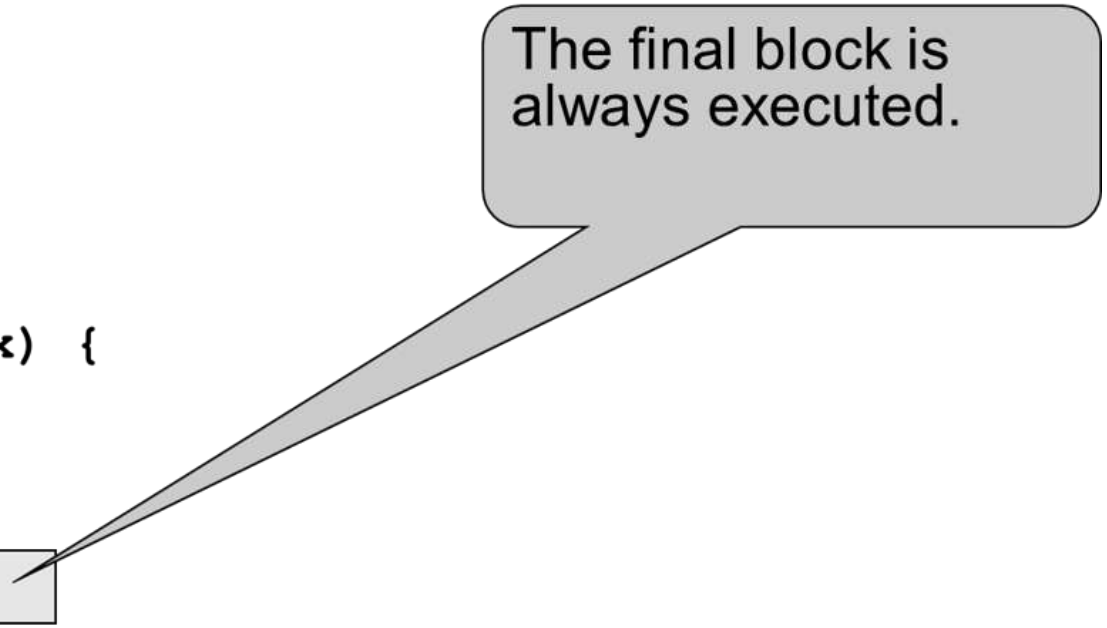
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (6 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```



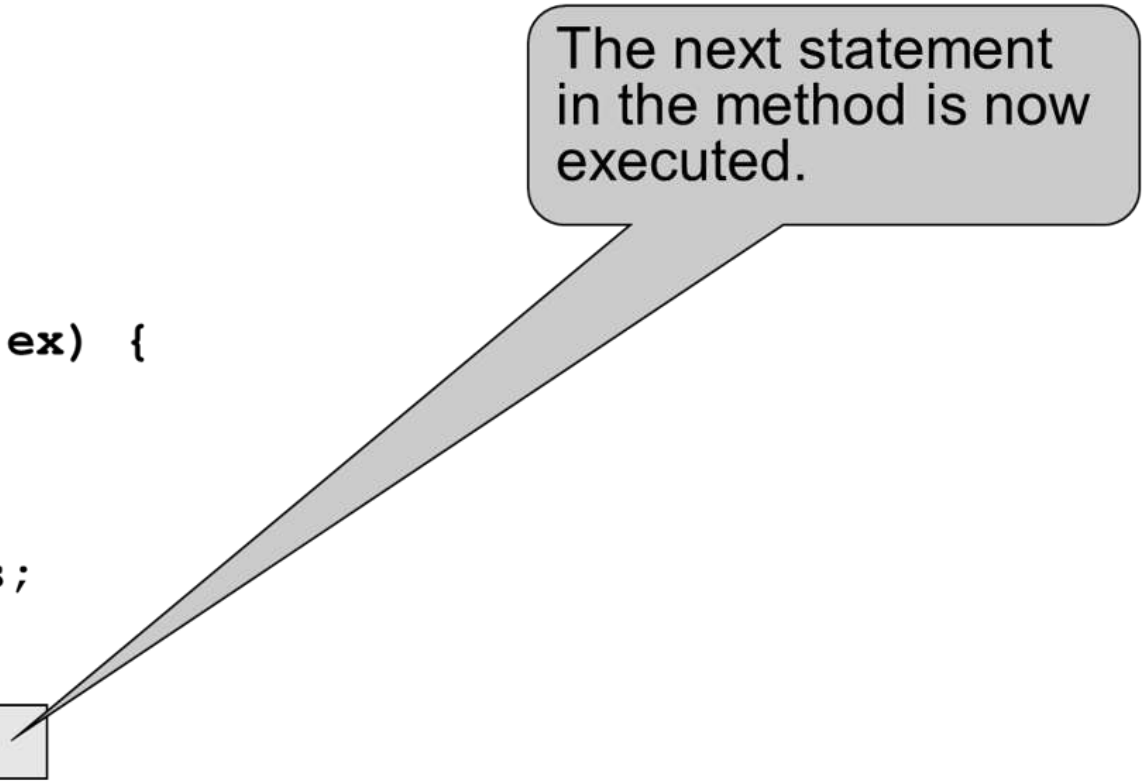
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (7 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```



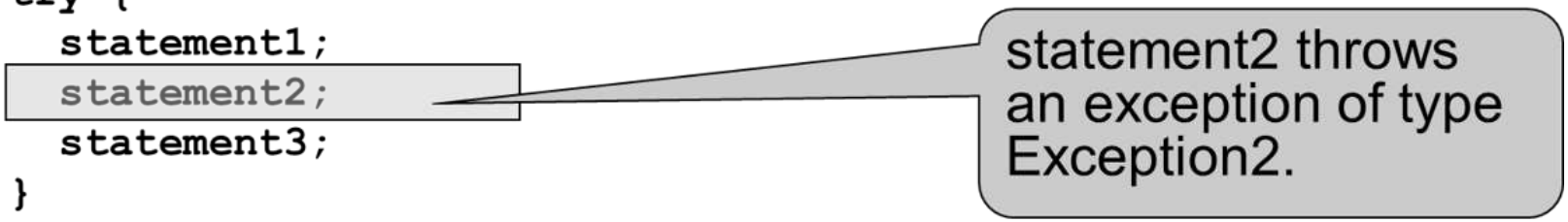
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (8 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



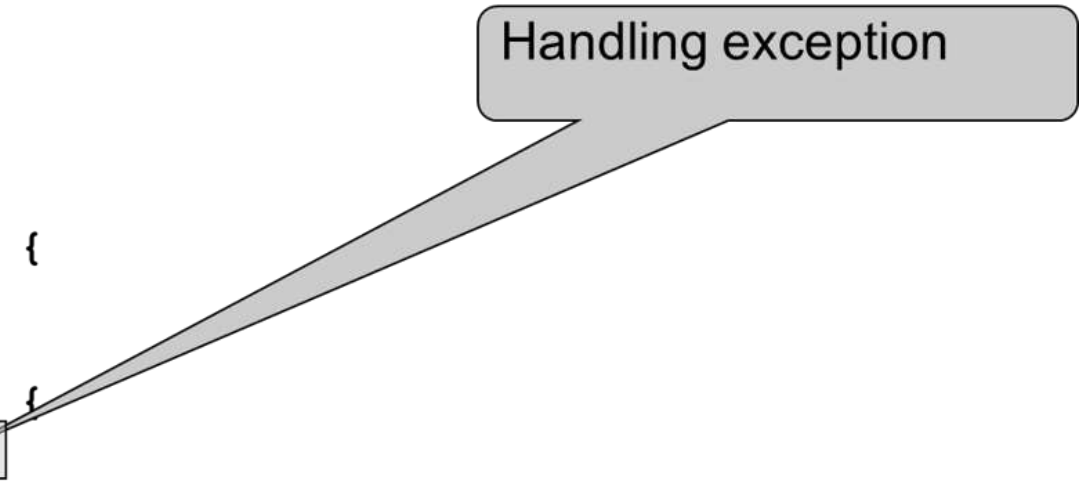
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (9 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



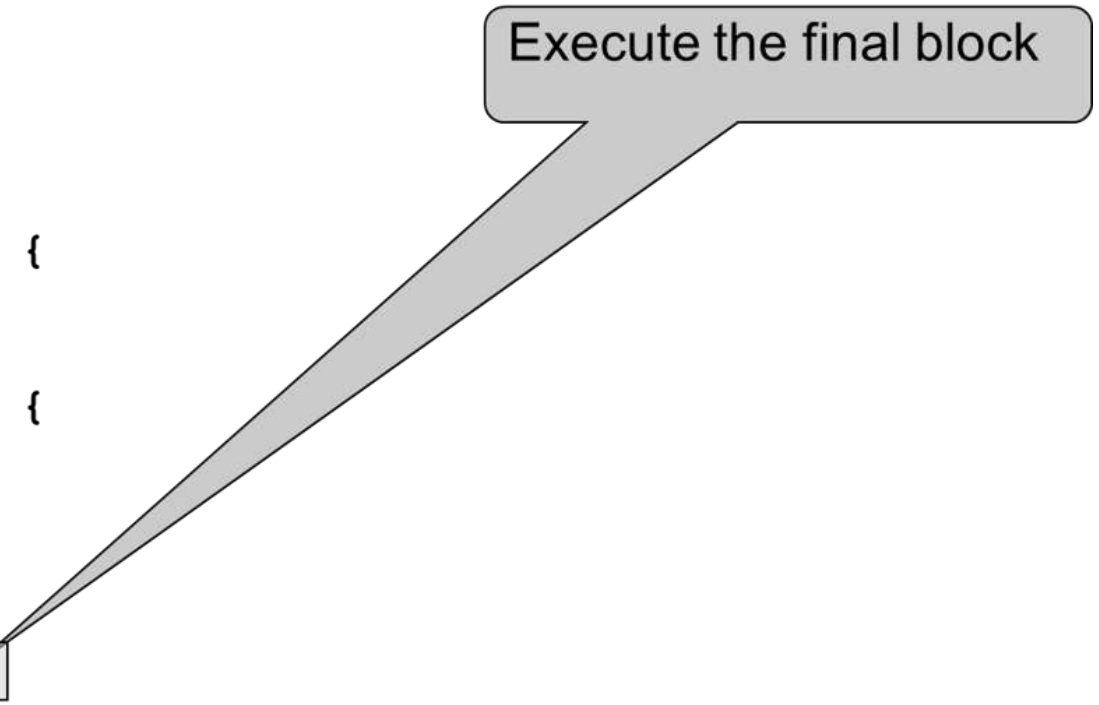
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (10 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



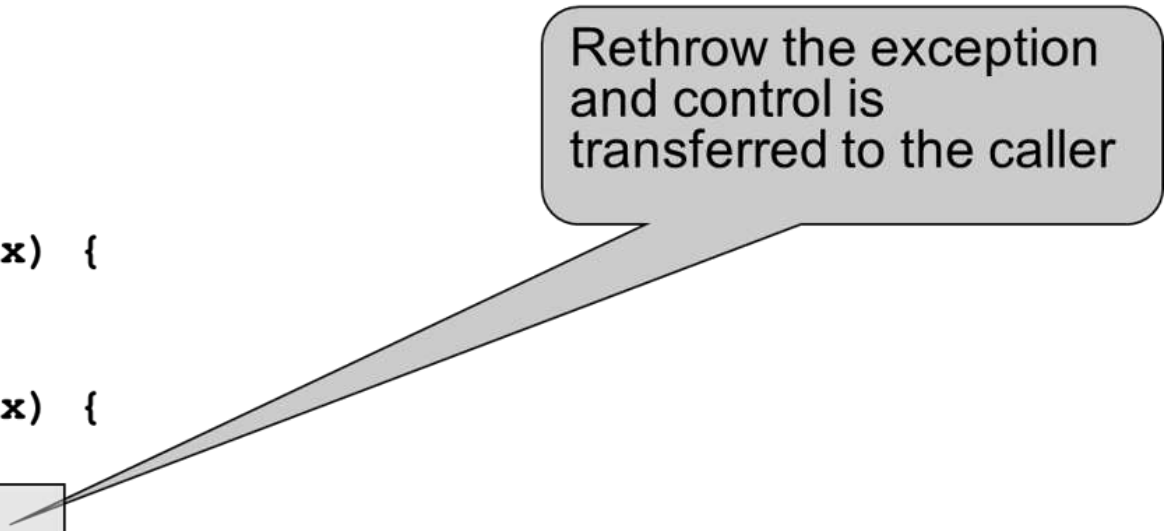
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace a Program Execution (11 of 11)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify. Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

When to Throw Exceptions

- An exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

When to Use Exceptions (1 of 2)

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {

  System.out.println(refVar.toString());

}

catch (NullPointerException ex) {

  System.out.println("refVar is null");

}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

When to Use Exceptions (2 of 2)

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Defining Custom Exception Classes

- Use the exception classes in the API whenever possible.
- Define custom exception classes if the predefined classes are not sufficient.
- Define custom exception classes by extending Exception or a subclass of Exception.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Custom Exception Class Example

In Listing 13.8, the **`setRadius`** method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler, you have to create a custom exception class.

InvalidRadiusException

CircleWithRadiusException

TestCircleWithRadiusException

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Assertions

An assertion is a Java statement that enables you to assert an assumption about your program. An assertion contains a Boolean expression that should be true during program execution. Assertions can be used to assure program correctness and avoid logic errors.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Declaring Assertions

An **`assertion`** is declared using the new Java keyword **`assert`** in JDK 1.4 as follows:

```
assert assertion; or
assert assertion : detailMessage;
```

where `assertion` is a Boolean expression and **`detailMessage`** is a primitive-type or an Object value.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Executing Assertions

When an assertion statement is executed, Java evaluates the assertion. If it is false, an `AssertionError` will be thrown. The `AssertionError` class has a no-arg constructor and seven overloaded single-argument constructors of type `int`, `long`, `float`, `double`, `boolean`, `char`, and `Object`.

For the first assert statement with no detail message, the no-arg constructor of `AssertionError` is used. For the second assert statement with a detail message, an appropriate `AssertionError` constructor is used to match the data type of the message. Since `AssertionError` is a subclass of Error, when an assertion becomes false, the program displays a message on the console and exits.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Executing Assertions Example

```
public class AssertionDemo {
  public static void main(String[] args) {
    int i; int sum = 0;
    for (i = 0; i < 10; i++) {
      sum += i;
    }
    assert i == 10;
    assert sum > 10 && sum < 5 * 10 : "sum is
    " + sum;
  }
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Compiling Programs With Assertions

Since **`assert`** is a new Java keyword introduced in JDK 1.4, you have to compile the program using a JDK 1.4 compiler. Furthermore, you need to include the switch –source 1.4 in the compiler command as follows:

```
javac –source 1.4 AssertionDemo.java
```

Note: If you use JDK 1.5, there is no need to use the –source 1.4 option in the command.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Running Programs With Assertions

By default, the assertions are disabled at runtime. To enable it, use the switch –enableassertions, or –ea for short, as follows:

```
java –ea AssertionDemo
```

Assertions can be selectively enabled or disabled at class level or package level. The disable switch is –disableassertions or –da for short. For example, the following command enables assertions in package **`package1`** and disables assertions in class **`Class1`**.

```
java –ea:package1 –da:Class1 AssertionDemo
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Using Exception Handling or Assertions (1 of 4)

Assertion should not be used to replace exception handling. Exception handling deals with unusual circumstances during program execution. Assertions are to assure the correctness of the program. Exception handling addresses robustness and assertion addresses correctness. Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks. Assertions are checked at runtime and can be turned on or off at startup time.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Using Exception Handling or Assertions (2 of 4)

Do not use assertions for argument checking in public methods. Valid arguments that may be passed to a public method are considered to be part of the method's contract. The contract must always be obeyed whether assertions are enabled or disabled. For example, the following code in the Circle class should be rewritten using exception handling.

```
public void setRadius(double newRadius) {
  assert newRadius >= 0;
  radius = newRadius;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Using Exception Handling or Assertions (3 of 4)

Use assertions to reaffirm assumptions. This gives you more confidence to assure correctness of the program. A common use of assertions is to replace assumptions with assertions in the code.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Using Exception Handling or Assertions (4 of 4)

Another good use of assertions is place assertions in a switch statement without a default case. For example,

```
switch (month) {
   case 1: ... ; break;
   case 2: ... ; break;
   ...
   case 12: ... ; break;
   default: assert false : "Invalid month: "
   + month
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `File` Class

The **`File`** class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion. The filename is a string. The **`File`** class is a wrapper class for the file name and its directory path.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Obtaining File Properties and Manipulating File

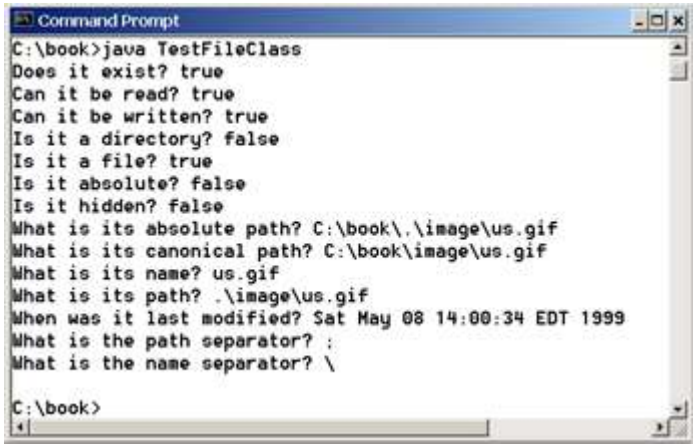
java.io.File	
`+File(pathname: String)`	Creates a `File` object for the specified path name. The path name may be a directory or a file.
`+File(parent: String, child: String)`	Creates a `File` object for the child under the directory parent. The child may be a file name or a subdirectory.
`+File(parent: File, child: String)`	Creates a `File` object for the child under the directory parent. The parent is a `File` object. In the preceding constructor, the parent is a string.
`+exists(): boolean`	Returns true if the file or the directory represented by the `File` object exists.
`+canRead(): boolean`	Returns true if the file represented by the `File` object exists and can be read.
`+canWrite(): boolean`	Returns true if the file represented by the `File` object exists and can be written.
`+isDirectory(): boolean`	Returns true if the `File` object represents a directory.
`+isFile(): boolean`	Returns true if the `File` object represents a file.
`+isAbsolute(): boolean`	Returns true if the `File` object is created using an absolute path name.
`+isHidden(): boolean`	Returns true if the file represented in the `File` object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
`+getAbsolutePath(): String`	Returns the complete absolute file or directory name represented by the `File` object.
`+getCanonicalPath(): String`	Returns the same as `getAbsolutePath()` except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
`+getName(): String`	Returns the last name of the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getName()` returns `test.dat`.
`+getPath(): String`	Returns the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getPath()` returns `c:\book\test.dat`.
`+getParent(): String`	Returns the complete parent directory of the current directory or the file represented by the `File` object. For example, new `File("c:\\book\\test.dat").getParent()` returns `c:\book`.
`+lastModified(): long`	Returns the time that the file was last modified.
`+length(): long`	Returns the size of the file, or 0 if it does not exist or if it is a directory.
`+listFile(): File[]`	Returns the files under the directory for a directory `File` object.
`+delete(): boolean`	Deletes the file or directory represented by this `File` object. The method returns true if the deletion succeeds.
`+renameTo(dest: File): boolean`	Renames the file or directory represented by this `File` object to the specified name represented in dest. The method returns true if the operation succeeds.
`+mkdir(): boolean`	Creates a directory represented in this `File` object. Returns true if the the directory is created successfully.
`+mkdirs(): boolean`	Same as `mkdir()` except that it creates directory along with its parent directories if the parent directories do not exist.

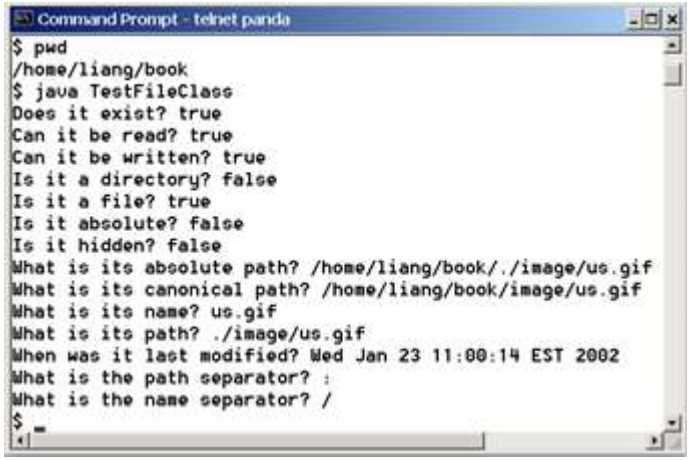
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Explore File Properties

Objective: Write a program that demonstrates how to create files in a platform-independent way and use the methods in the File class to obtain their properties. The following figures show a sample run of the program on Windows and on Unix.





TestFileClass

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Text I/O

A **`File`** object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the **`Scanner`** and **`PrintWriter`** classes.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Writing Data Using `PrintWriter`

java.io.PrintWriter	
+PrintWriter(filename: String)	Creates a PrintWriter for the specified file.
+print(s: String): void	Writes a string.
+print(c: char): void	Writes a character.
+print(cArray: char[]): void	Writes an array of character.
+print(i: int): void	Writes an int value.
+print(l: long): void	Writes a long value.
+print(f: float): void	Writes a float value.
+print(d: double): void	Writes a double value.
+print(b: boolean): void	Writes a boolean value.
Also contains the overloaded println methods.	A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is \r\n on Windows and \n on Unix.
Also contains the overloaded printf methods.	The printf method was introduced in §4.6, "Formatting Console Output and Strings."

WriteData

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Try-With-Resources

Programmers often forget to close the file. JDK 7 provides the followings new try-with-resources syntax that automatically closes the files.

```
try (declare and create resources) {
 Use the resource to process the file;
}
```

WriteDataWithAutoClose

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Reading Data Using `Scanner`

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

ReadData

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Replacing Text

Write a class named **ReplaceText** that replaces a string in a text file with a new string. The filename and strings are passed as command-line arguments as follows:

```
java ReplaceText sourceFile targetFile oldString newString
```

For example, invoking

```
java ReplaceText FormatString.java t.txt StringBuilder
  StringBuffer
```

replaces all the occurrences of **StringBuilder** by **StringBuffer** in FormatString.java and saves the new file in t.txt.

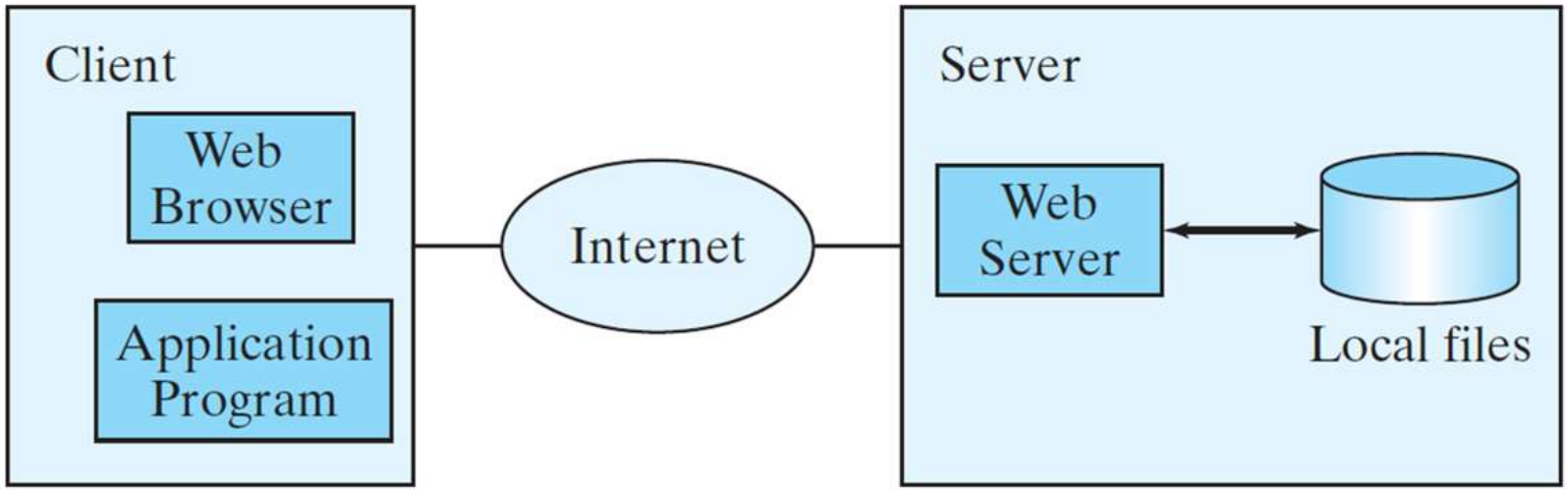
ReplaceText

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Reading Data From the Web (1 of 2)

Just like you can read data from a file on your computer, you can read data from a file on the Web.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Reading Data From the Web (2 of 2)

```
URL url = new URL("www.google.com/index.html");
```

After a **URL** object is created, you can use the **openStream()** method defined in the **URL** class to open an input stream and use this stream to create a **Scanner** object as follows:

```
Scanner input = new Scanner(url.openStream());
```

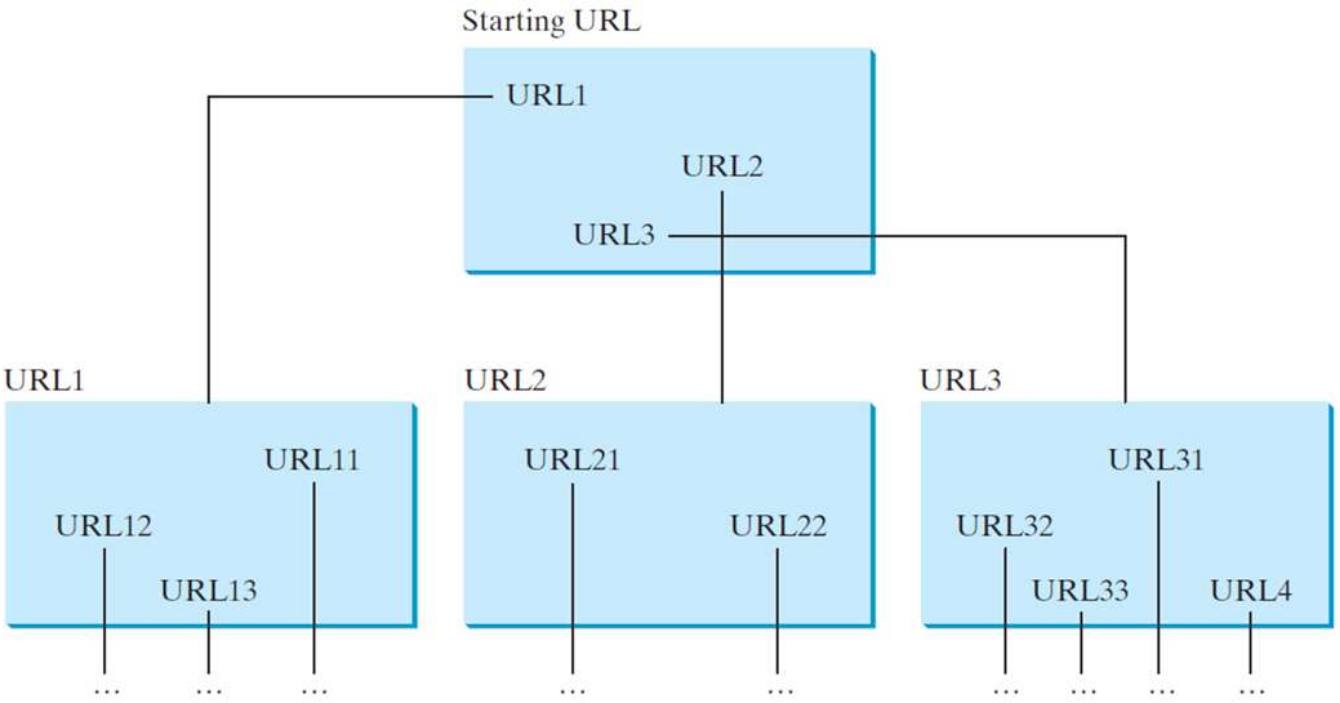
ReadFileFromURL

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Web Crawler (1 of 3)

This case study develops a program that travels the Web by following hyperlinks.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Web Crawler (2 of 3)

The program follows the URLs to traverse the Web. To avoid that each URL is traversed only once, the program maintains two lists of URLs. One list stores the URLs pending for traversing and the other stores the URLs that have already been traversed. The algorithm for this program can be described as follows:

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Web Crawler (3 of 3)

Add the starting URL to a list named `listOfPendingURLs;`

```
while listOfPendingURLs is not empty {
   Remove a URL from listOfPendingURLs;
   if this URL is not in listOfTraversedURLs {
     Add it to listOfTraversedURLs;
     Display this URL;
     Exit the while loop when the size of S is equal to 100.
     Read the page from this URL and for each URL contained in the page {
      Add it to listOfPendingURLs if it is not is listOfTraversedURLs;
    }
  }
}
```

WebCrawler

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved